also identify when a new threat model is needed.

Do It Yourself

Much of the language in this guide describes how Application Security Stable Counterparts should threat model with teams, but that doesn't have to be the case! Security is everyone's responsibility and you can threat model too! Reach out to sec-appsec on Slack if you need help.

SECTION: security/product-security/data-security/_index.md

Team Identity

GitLab's Data Security team is responsible for investigating and remediating issues in data security and privacy across our product. We supplement the already excellent work being done by our colleagues in Product Security as well as all of Engineering by taking (often broad and vague) concerns, investigating them to find what (if anything) needs to be remediated, and then working to assist in cleanup. To put it another way, we dot i's and cross t's across GitLab.

The Data Security Team's remit includes (but isn't sharply limited to):

Data stored out of compliance with our data classification standard or our associated infrastructure standards
Secrets and encryption keys stored outside KMS or HSM systems
Copies of RED data outside official production systems
Excessive permission grants in production systems that give the power to a single person to rm -rf gitlab.com (or similar)

Some high-priority initiatives for the team are:

We're brand new (we began in August 2024)
So we're working on these.
Come back soon!

Team Members

<table>
<thead>
<tr>
<th>Person</th>
<th>Role</th>
</tr>
</thead>
<tbody>
<tr>
<td>Julie Davila</td>
<td>VP, Product Security</td>
</tr>

<tr>
<td>Jacob Jernigan</td>
<td>Senior Manager, Infrastructure Security</td>
</tr>
<tr>
<td>Justin Cameron</td>
<td>Staff Security Engineer, Data Security</td>
</tr>
<tr>
<td>Brendan O'Connor</td>
<td>Staff Security Engineer, Data Security</td>
</tr>
</tbody>
</table>

Working With Us

Know about something scary and weird?

To ask us to look into an area of potential scariness, please create an issue using our something weird template.

What We'll Do

We will look at every issue as it comes in. We may not respond to every issue immediately. We will investigate to the best of our abilities (including partnering with other teams to understand systems better). We'll try to get to the bottom of what's happening and work with the teams who are best-placed to fix the issue if there is one.

As our processes develop, we'll add more status labels and the like to our issues so it's easier for other teams to see what we're working on.

Confidentiality

In keeping with our value of Transparency, this issue tracker is open to everyone at GitLab. If you feel a need to ask us to look into an issue in a way that will not be tagged with your identity, please DM someone on the team and have them create the issue for you.

Anything else

Create an issue in our tracker.
Tag us on an issue (anywhere) using @gitlab-com/gl-security/product-security/data-security to mention the whole team.
Come say hello in Slack in our security-datasec channel. Feel free to share an amusing meme!

How We Work

We'll add this once we know more.

SECTION: security/product-security/infrastructure-security/_index.md

Team Identity

GitLab's Infrastructure Security team is responsible for the planning, execution, and support of initiatives specific to the security of GitLab's Infrastructure.

As stable counterparts from the Security department, the team members support specific Product Categories heavily reliant on Cloud and Infrastructure solutions. They collaborate with Development, Infrastructure, and Security teams across various product categories.

Infrastructure Security's engagements take place in the form of infrastructure change reviews, SaaS infrastructure access & permissions models, cloud security best practices, operating system security, security monitoring at the host and container level, vulnerability management, and patching policies.

Some high-priority initiatives for the team are:

- Deployment of tools to improve our data collection capabilities (e.g., Wiz, OSQuery)
- Counterpart work (e.g., Dedicated, FedRAMP, Cells, AI)
- Security reviews (e.g., Readiness reviews, Design reviews)
- Identify and remediate Misconfiguration across our Cloud Environments
- Creation and deployment of preventive controls (e.g., AWS/GCP Organization hardening, Terraform hardening)

Further details can be found in the job family description.

Team Members

<table>
<thead>
<tr>
<th>Person</th>
<th>Role</th>
</tr>
</thead>
<tbody>
<tr>
<td>Julie Davila</td>
<td>VP, Product Security</td>
</tr>
<tr>
<td>Jacob Jernigan</td>
<td>Senior Manager, Infrastructure Security</td>

</tr>
<tr>
<td>Dennis Salzmann</td>
<td>Manager, Infrastructure Security</td>
</tr>
<tr>
<td>Matt Morrison</td>
<td>Staff Security Engineer, Infrastructure Security</td>
</tr>
<tr>
<td>Dhruv Jain</td>
<td>Senior Security Engineer, Infrastructure Security</td>
</tr>
<tr>
<td>Uday Govindia</td>
<td>Senior Security Engineer, Infrastructure Security</td>
</tr>
<tr>
<td>Lizzie Moratti</td>
<td>Intermediate Security Engineer, Infrastructure Security</td>
</tr>
<tr>
<td>Justin Shields</td>
<td>Intermediate Security Engineer, Infrastructure Security</td>
</tr>
<tr>
<td>Yang Lyu</td>
<td>Intermediate Security Engineer, Infrastructure Security</td>
</tr>
</tbody>
</table>

Working With Us

1. To request an infrastructure security review, please create an issue using the security review template
1. For everything else:
    1. Create an issue in our issue tracker dedicated to Business as Usual (BAU) activities and general inquiries.
        - It is not necessary to @mention anyone. In case you want to mention the whole team, use the @gitlab-com/gl-security/product-security/infrastructure-security handle on GitLab.com.
        - You can also chat with us on Slack in the dedicated security-infrasec channel or by tagging us @infrasec-team.

2. The team will triage (and prioritise accordingly) all incoming request during the fortnightly team sync (typically Tuesday).

How We Work

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our project issue tracker as described in the project management section below.

The team does have set of regular synchronous calls:

- A fortnightly team sync to discuss progress, blockers, and anything related to the InfraSec team.
  - Everyone in the company is welcome to join.
  - The agenda is public within GitLab as well.
- A quarterly team retrospective to reflect on what went well in the previous quarter, and discuss what can be improved going forward.
- 1-1s between Individual Contributors and the Engineering Manager.

Team Pages

- This Handbook Page, which contains general information about the team
- The Internal Handbook, which is the operational source of truth for the team. Everyone is encouraged to check it out for team's information
- The Infrastructure Security GitLab Sub-Group, which contains EPICs and repositories
- The Infrastructure Security Public Sub-Group, which contains publicly facing resources (e.g., Docker images, etc.)

Project Management

We use Epics, Issues, and Issue Boards to organize our work, as they complement each other:

- The single source of truth for engineering work is the InfraSec Sub-Group in GitLab. All Epics will be collected at this level
- Having all projects at this level allows us to use a single list for prioritization and enables us to prioritize work for different services alongside each other
- Projects are prioritized in line with the InfraSec Goals

Team Planning

- For the long term strategy of the InfraSec Team, you can refer to:
  - · InfraSec Goals
- From a tactical point of view, you can refer to:
  - · InfraSec Planning Board (for the tasks we are currently working on)

Project Ownership

Each project has an owner who is responsible for delivering the project.

The owner needs to:

1. Regularly update the status in the Epic description and milestones.
1. Work with others to move project issues through the boards.

Labels

Please use the following labels for project work only:

Label	Use Case
~"Department::InfraSec"	Team Label (to be included in every project-related issue)
~"InfraSec::triage"	For new issues which need to be triaged
~"InfraSec::this-quarter"	For EPICs committed to the current quarter

Design Documents

Before starting a new project, the team is encouraged
to define software designs through design docs.
These design doc documents the high level implementation strategy and key design decisions with emphasis on the trade-offs that were considered during those decisions.

To start discussing a new design:

1. Create a new MR in the InfraSec Team Charter repo with the Design proposal. You can use this template as a reference for the structure of the Design doc.
2. Fill the data as requested
3. Mark other members of the team as reviewers

Additional Resources

Onboarding

- Infrastructure Security Team Onboarding Template
- InfraSec Entitlements template

SECTION: security/product-security/infrastructure-security/issue-lifecycle.md

Managing issues·from creation to closure·is a fundamental process that ensures they are addressed systematically. The issue lifecycle typically begins when a team member or stakeholder creates an issue, detailing the problem, enhancement, or task to be completed. The issue progresses through various stages, including triage and in-progress, before being resolved or closed. This workflow allows for transparent tracking of work, accountability among team members, and a structured approach to delivering solutions. Each stage is crucial to ensuring that issues are handled efficiently, fostering collaboration and addressing all concerns on time.

Issue Stages

Stage 1: Issue Creation

The first step is to ensure it is properly categorized for review by the InfraSec team.

- Initially, standard labels like Department::InfraSec are applied if they are not already present to indicate that the issue pertains to the Infrastructure Security team.
- After the above step, the label InfraSec::triage is applied to the issue. This label signals to the InfraSec team that the issue requires their attention and triage.
- During the triage process, the InfraSec team assesses the issue's priority, scope, and impact on the infrastructure.
- Based on this assessment, the issue is either moved to the InfraSec::backlog for future consideration or marked as "InfraSec::prioritised" if it requires immediate action.
- This structured approach ensures that issues are addressed according to their urgency and business impact.

Stage 2: Working on Issues

- Once an issue has been marked as InfraSec::prioritised during triage, it moves into the second stage, where the InfraSec team begins actively working on it.
- At this point, the issue is updated with the label InfraSec::WIP, signaling that the team has started addressing the problem or task.
- The team also estimates the effort required to resolve the issue. This is done by applying an effort estimation label, such as InfraSecWork::Large, to indicate the size and complexity of the work involved. For more information, refer to Infrastructure Security - Capacity Indicators and Workflows.
- This estimation helps with setting realistic timelines and aligning expectations.

Stage 3: Backlog Checks

- In this stage, the InfraSec team regularly reviews issues in the InfraSec::backlog to ensure they remain relevant and necessary.
- If an issue has been in the backlog for more than 6 months without any progress or it is no longer relevant, it is reviewed for potential closure. The issue is tagged as InfraSec::stale, and the triage bot will close it if it is not closed.
- If an issue has already been marked as InfraSec::prioritised or moved to the InfraSec::WIP stage in Stage 2, this stage is automatically skipped, as the issue is actively being worked on.
- This step helps maintain a clean backlog and focus on issues that are still pertinent to the team's objectives.

Stage 4: Issue Closure

- In the final stage, after the InfraSec team has completed work on an issue and met the defined success criteria, the issue is prepared for closure.
- The team first removes the InfraSec::WIP label, indicating the work has been completed.
- After verifying that all success criteria and objectives have been achieved, the issue is then officially closed.
- This stage ensures that the work has been successfully completed and documented, marking the

conclusion of the issue's lifecycle.

Handling issues using TriageBot

This might look like a very complex workflow, but we have a friendly bot that helps you set the right labels. The InfraSec team utilizes the GitLab Triage Bot to automate the initial handling of security-related issues. By leveraging this bot, issues are automatically categorized, labeled, and assigned according to predefined criteria, ensuring efficient prioritization and management throughout the issue lifecycle.

Configuration of the bot for InfraSec use-cases is available here.

When an issue is created

```
mermaid
flowchart TD
    A[TriageBot] --> B{Does the issue has 'InfraSec::triage' label?}
    B -->|No| C(Apply the 'InfraSec::triage' label)
    A --> D{Does the issue has standard labels such as 'Department::InfraSec'?}
    D --> |No| E(Apply the standard labels)
    D --> |Yes| F{Is the issue assigned to anyone?}
    F --> |No| G(Tag Infrasec team and ask them to review the issue)
```

Issue closure check

```
mermaid
flowchart TD
    A[TriageBot] --> B{Does the issue has 'InfraSec::backlog' label?}
    B --> C{Is the issue older than 3 months?}
    C --> |Yes| D{Tag assignee and comment if the isue is still relevant}
    B --> E{Is the issue older than 6 months?}
    E --> |Yes| F(Add the label 'InfraSec::stale' to the issue)
    F --> |Yes| G(Tag assignee and inform of the 'InfraSec::stale' label)

    A --> H{Does the issue has 'InfraSec::stale' label?}
    H --> |Yes| I(Close the issue)

    A --> J{Does the issue has 'InfraSec::WIP' label?}
    J --> |Yes| K{Is the issue closed?}
    K --> |No| L(Continue to the next chart)
    K --> |Yes| M(Remove the 'InfraSec::WIP' label)
```

WIP Issue checks

```
mermaid
flowchart TD
    I[TriageBot] --> A{Is the issue open and has 'InfraSec::WIP' label?}
    A --> |No| B{Does the issue has a due date?}
```

B --> |No| C(Tag assignee and comment that issue is missing the due date)

A --> |No| D{Does the issue has weightage assigned?}
D --> |No| E(Tag assignee and comment that the issue is missing the Efforts scoping label)

A --> |No| F{Does the issue has a description?}
F --> |No| G(Tag assignee and comment that issue is missing the description)

SECTION: security/product-security/infrastructure-security/metrics/capacity.md

Overview

This page provides an overview of how we measure the team's capacity to effectively handle current workloads and plan for future needs. By collecting and analyzing this data, we can make informed decisions regarding the team's capacity and headcount requirements.

Additionally, this page includes information on the type of work classification and effort classification. The type of work classification helps us identify areas where additional capacity or other changes may be necessary. It provides a detailed breakdown of different types of work associated with the InfraSec team, such as stable counterpart duties, threat model duties, merge request reviews, and more.

The effort classification, on the other hand, provides an estimate of the level of effort required to resolve a task. It is not a measure of the actual time spent on the task. The effort classification is categorized into different weights, ranging from trivial to too large, and serves as a guideline for understanding the complexity and time commitment associated with each task.

Collecting this data helps inform decisions involving the team's capacity and headcount needs. These metrics are only analyzed in aggregate and are not utilized or referenced for evaluating individual team member performance. They are solely used to understand overall team dynamics and requirements.

Where are the charts that are based on this data?

This still needs to be done and is tracked in this issue.

Metrics Review Frequency

The metrics review occurs at the beginning of each month. Led by InfraSec, this review involves a comprehensive analysis and discussion of the metrics. It is a valuable opportunity to assess the team's capacity and make data-driven decisions. For the review, an issue will be created which will have a due date of 7 days for team members to provide their feedback. The feedback from the team will be discussed in the next InfraSec Sync that happens.

During the review, we examine key metrics related to capacity, workload, and effort classification. By analyzing this data, we gain insights into the team's capacity, identify areas for improvement, and ensure that we have the necessary people to meet our security goals.

Work Classification

Classifying each type of work helps to distinguish where exactly more capacity or other changes may be necessary. For that, we utilize the following GitLab labels, which are created/managed/used at the gitlab-com group level.

Label	Description
InfraSecWork::Counterpart	Indicates the work is associated with the InfraSec stable counterpart duties
InfraSecWork::Investigation	Indicates the work is related to investigation-related issues
InfraSecWork::Maintenance	Indicates the work is associated with InfraSec maintenance activities
InfraSecWork::Project	Indicates the work is related to an InfraSec project
InfraSecWork::Discussion	Indicates an ongoing Discussion within the Infrasec team

Who assigns this label and when?

The DRI is expected to assign this label to any new Issue.

Effort Classification

The effort classification provides an estimate of the level of effort required to resolve a task, serving as a guideline rather than an actual measure of time spent. The Estimation Guide offers a reference point for understanding the complexity and time commitment associated with each task.

To provide further clarity, the effort classification is categorized into different weights, ranging from trivial to too large. These weights help understand the level of effort required, with trivial tasks requiring very little effort and too large tasks being highly complex and time-consuming. The Estimation Guide provides a rough estimate of the time commitment associated with each weight, helping to inform planning and resource allocation decisions.

Label	Weight	Classification	Description	Estimation Guide
InfraSecEffort::Trivial	1	Trivial	Very little effort required	Immediate or near immediate change to resolve the issue. This label should be used when effort is less than 1 day
InfraSecEffort::Small	2	Small	Straight forward change, minimal investigation	1-2 days
InfraSecEffort::Medium	3	Medium	Some investigation and/or collaboration needed	< 1 week
InfraSecEffort::Large	5	Large	Significant investigation and collaboration needed	1-2 weeks
InfraSecEffort::Too Large	13	Needs Refinement	The issue is overly complex and needs to be promoted to an Epic or broken down into smaller issues	N/A

Labelling of the issues should be done as follows:

- For issues which are to be labelled InfraSecEffort::Medium or less, the DRI will do it

directly based on their own rationale and estimation.
- For issues which are to be labelled InfraSecEffort::Large or InfraSecEffort::Too Large, peer review will be done and issues will be labelled accordingly

Handling Reestimation/blocks

- If an issue was estimated wrong, update the InfraSecEffort label and add another label InfraSec::Reestimated. This will help us improve the process.
- When the task is blocked due to a dependency on an external team, add InfraSec::Blocked label and remove the estimation. This will help us to identify tasks that are blocked on other teams and help us to track them more easily as well.

Handling shift in priority

- If a task is deprioritised for a long period - We move the task to backlog by adding label InfraSec::backlog and update the label to show how much work has been done. For example, For an estimation of InfraSecEffort::Large, if the work was done in the timeframe for InfraSecEffort::Medium, we change the label and shift it to the backlog. The capacity has been measured this way. Next time we pick it up, we can reassess the estimation.
- If a task can be picked up after a few days, there is no need to do anything.

Who assigns this label and when?

The DRI is expected to assign this label to any new Issue.

SECTION: security/product-security/security-logging/_index.md

 Our Vision

Guarantee that GitLab has the logging data coverage required to:

- Perform the threat analysis, alerting and threat detections necessary to protect the company and its customers
- Ensure compliance with internal policies, standards, internal and external audits, and regulatory requirements.

Our Mission Statement

The team achieves its vision by planing, executing and supporting initiatives
that improve the coverage and usability of security logging data on GitLab.
We manage, maintain, design, configure, and document the necessary tools,
systems and processes to make that happen.

Further details can be found in the job family description.

 The Team

Team Members

The Security Logging Team is part of the Product Security sub-department. See GitLab's organizational chart and meet our team members.

Team Responsibilities

The Security Logging Team is responsible for security focused logging, monitoring, and alerting.

What we are responsible for

The Security Logging Team is responsible for managing, maintaining, designing, configuring, and documenting the necessary tools, systems and processes to support all security logging, monitoring, and alerting needs. This includes but is not limited to the following examples:

- Designing, managing, and maintaining the security logging and monitoring infrastructure
- Ensuring reliability and availability of log data for security purposes
- Owning and maintaining the security logging standard that defines important aspects and requirements of security logging, monitoring, and alerting at GitLab
- Working with our internal GitLab customers to ensure they have the logging data, and access to this data, needed to successfully accomplish the responsibilities of their roles
- Working closely with the Infrastructure team ensuring that log aggregation infrastructure is reliable and available to meet the needs of both Infrastructure and Security
- Building an maintaining a library of logging profiles for various technologies that can used to gather, format, and send logging data to the appropriate log aggregation systems

What we are not responsible for

The Security Logging Team is not responsible for the logging, monitoring, and alerting data or infrastructure supporting non-security focused needs. This includes but is not limited to the following examples:

- Infrastructure or application logging supporting reliability or availability of GitLab systems
- Responding to or taking action on the security alerts that are generated by the infrastructure and tools owned and maintained by the Security Logging Team
- Remediation of security vulnerabilities found using the infrastructure and tools owned and maintained by the Security Logging Team
- The Security Logging Team does not own the logging data aggregated, stored, or contained within the infrastructure and tools owned and maintained by the Security Logging Team
- The Security Logging Team is not responsible for the endpoint systems from which logging data is gathered

Working With Us

1. Create an issue in our issue tracker dedicated to Business as Usual (BAU) activities and general inquiries.
    - It is not necessary to @mention anyone. In case you want to mention the whole team, use the @gitlab-com/gl-security/engineering-and-research/security-logging handle on GitLab.com.
    - You can also chat with us on Slack in the dedicated security-logging channel or by tagging us @security-logging-team.

How to contact us

The Security Logging Team can be contacted in Slack using the security-logging channel, the security channel, or the security-division channel. You can also contribute, comment, view, or interact with us in our team repo.

How We Work

We are an internal customer focused and customer driven team. Our customers drive our priorities and help us define our responsibilities. We work to balance this with a risk based approach aimed at reducing and minimizing security risk at GitLab. Additionally, we embrace the DevOps model, software defined infrastructures, a cloud first approach, modular decoupled architectures, self-serviceability, and automate when and wherever possible.

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our project issue tracker as described in the project management section below.

The team does have set of regular synchronous calls:

- A weekly team sync to discuss progress, blockers, and anything related to the InfraSec team.
  - Everyone in the company is welcome to join.
  - The agenda is public within GitLab as well.
- A quarterly team retrospective to reflect on what went well in the previous quarter, and discuss what can be improved going forward.
- 1-1s between Individual Contributors and the Engineering Manager.

Team Pages

- This Handbook Page, which contains general information about the team.
- The Security-Logging GitLab Sub-Group, which contains EPICs and repositories

Project Management

We use Epics, Issues, and Issue Boards to organize our work, as they complement each other:

- The single source of truth for engineering work is the Security-Logging Sub-Group in GitLab. All Epics will be collected at this level.
- Having all projects at this level allows us to use a single list for prioritization and enables us to prioritize
work for different services alongside each other.
- Projects are prioritized in line with Security Logging Board.

Team Planning

- For the long term strategy of the InfraSec Team, you can refer to:
  - · Sec-Logging Roadmap (tbd)
  - · Sec-Logging OKRs (tbd)
- From a tactical point of view, you can refer to:

- · Sec-Logging Milestones (quarterly) (tbd)
- · Sec-Logging Epics for this quarter (tbd)
- · Sec-Logging Initiatives Board (tbd) (for the tasks we are currently working on)

Project Ownership

Each project has an owner who is responsible for delivering the project.

The owner needs to:

1. Regularly update the status in the Epic description and milestones.
1. Work with others to move project issues through the boards.

Labels

Please use the following labels for general work only:

Label	Use Case
~"·· SecLog"	Team Label (to be included in every project-related issue)
~"SecLog::Incoming-Requests"	For new issues which need to be triaged

Design Documents

Before starting a new project, the team is encouraged
to define software designs through design docs.
These design doc documents the high level implementation strategy and key design decisions with emphasis on the trade-offs that were considered during those decisions.

To start discussing a new design:

1. Create a new issue in the InfraSec Team Charter repo
1. Select the designdoc template
1. Fill the data as requested

Security Logging Program Roles and Responsibilities

The following roles and responsibilities are specific to the management and execution of the Security Logging Program which is overall the responsibility of the Product Security sub-department.

Security Logging is responsible for

- Ownership of, management, and maintenance of our SIEM
- Coordinating and executing log source ingestion
- Overall management and coordination of the security logging program
- Ownership of the to be developed Security Logging Standard

- Subject matter experts with regards to the operation and management of the SIEM
- The data movement and archive of security logging data currently held in Panther
- Capacity planning and forecasting of licensing and infrastructure costs, as a shared responsibility with InfraSec

AppSec is responsible for

- Working closely with Development to ensure the Security Logging Standard is adopted and followed
- Advising SecLogging when log formats will change, prior to the change occurring, if known
- Advising SIRT on any log sources from the application that they may be interested in and recommend new high-value detection considerations to SIRT for new product features or capabilities
- Helping to bridge the gap between the Security Logging Team and the software engineering teams
- Helping to identify logging gaps in the application and proposing solutions to close the gaps
- Aiding and guiding engineering teams that design, develop, or deploy something new on what to log and how to get the logs into the SIEM. This includes logs that SIRT needs to complete investigations.

InfraSec is responsible for

- Working closely with Infrastructure to ensure the Security Logging Standard is adopted and followed
- Advising SecLogging when log formats will change, prior to the change occurring, if known
- Advising SIRT on any log sources from the infrastructure that they may be interested in and recommending new high-value detection considerations to SIRT for new product features or capabilities
- Helping to bridge the gap between the Security Logging Team and the Infrastructure teams
- Helping to identify logging gaps in the infrastructure and proposing solutions to close the gaps
- Aiding and guiding engineering teams that design, develop, or deploy something new on what to log and how to get the logs into the SIEM. This includes logs that SIRT needs to complete investigations.

Capacity planning and forecasting of licensing and infrastructure costs, as a shared responsibility with Security Logging

SIRT is responsible for

- The management and ownership of Panther until it is decommissioned
- Ensuring that they have the logging data needed to effectively and efficiently execute their responsibilities as incident responders
- Documenting gaps in logs and using the prioritization procedure to ensure prioritization meets the standards set by the procedure
- Configuring automation, alerting, and monitoring specific to their needs for incident response
- Guiding and informing the InfraSec and AppSec teams on logging data critical to SIRT through collaboration and maintenance of the development of the Security Logging Standard
- Reporting to the Security Logging Team when logs:
  - Are no longer needed

- Are incomplete
- Need format improvements or corrections
- Require more verbosity
- Have efficiency opportunities (e.g. we only use a small portion of sizable logs)

Everyone else across Security is responsible for

- Making requests of the Security Logging Team for logs that you need to execute your responsibilities, we will have a documented process for this soon
- Submitting issues to SIRT when you have detection suggestions where we have a high return on investment

Additional Resources

Onboarding

- Infrastructure Security Team Onboarding Template
- InfraSec Entitlements template

SECTION: security/product-security/security-logging/critical-logging-methodology.md

Purpose

The purpose of the Critical Logging Tiering Methodology is to support GitLab in categorizing logs based on their effect on GitLab's SaaS subscriptions and the achievement of GitLab's mission and goals. Ultimately, this provides GitLab with a mechanism to take a proactive approach to comprehensive risk management which considers risks, such as information security and privacy risks, impacting business operations across the organization. Additionally, by classifying logging into specific tiers, GitLab will be in a better position to appropriately prioritize risk mitigation activities and tailor internal controls based on a log's related tier.

Scope

The Critical Logging tiering methodology is applicable to all systems utilized across GitLab which are tracked within the Tech Stack to ensure that all systems are vetted completely and accurately using a consistent and standardized methodology.

Roles and Responsibilities

Role	Responsibility
Security Logging Team	Executes an annual review of Critical Logging tiers and makes adjustments as necessary. Owns the Critical Logging Tiering Methodology and supports the identification of and assignment of a Critical Logging tier as needed.
SIRT	Supports defining of Critical Logging tier in conjunction with the Security Logging Team when new systems are added to the tech stack.
AppSec	Supports defining of Critical Logging tier in conjunction with the Security Logging

Team when new systems are added to the tech stack.|
|InfraSec|Supports defining of Critical Logging tier in conjunction with the Security Logging Team when new systems are added to the tech stack.|
|Technical System Owners|Provide complete and accurate data about the systems that they own so that an accurate tier is assigned.|

Critical Logging Tiering Procedure

Defining what Critical Logging means at GitLab can be complex given the nature of our environment and the amount of integrations that exist across the many systems that are used to carry out business activities. As part of GitLab's Business Impact Analysis (BIA) process, we obtain inputs that assist in the assignment of a critical system tier per system. These inputs used to determine system criticality tiers include, but are not limited to, the following:

1. If the system was compromised, would there be an immediate impact to GitLab.com SaaS subscriptions
1. If the system was compromised, which would describe the impact to GitLab:
   - Critical business functions (i.e., indirectly affects revenue generation, requires constant availability for effective business operation)
   - Operational business functions (i.e., affects efficiency/cost of operation)
   - Administrative functions (i.e., affects GitLab team members only on an individual basis (e.g., quality of life, individual productivity))

Once the information is obtained, it is reviewed by the Security Risk and/or IT Compliance Team to determine which system tier should be assigned to the system.

Then the Security Logging Team with the support of SIRT/AppSec/InfraSec takes the output of the findings above and applies the guidelines described below in Determining Critical Logging Tiers to each log source.

Determining Critical Logging Tiers

Systems are assigned a Critical Logging tier based on the following matrix:

<style type="text/css">
.tg {border-collapse:collapse;border-spacing:0;margin:0px auto;}
.tg td{border-color:black;border-style:solid;border-width:1px;overflow:hidden;padding:10px 5px;word-break:normal;}
.tg th{border-color:black;border-style:solid;border-width:1px;overflow:hidden;padding:10px 5px;word-break:normal;}
.tg .tg-zqun{background-color:ffffff;color:000000;text-align:center;vertical-align:middle}
.tg .tg-knp3{background-color:6e49cb;border-color:000000;color:ffffff !important;;
  text-align:center;vertical-align:middle}
.tg .tg-clye{background-color:380d75;color:ffffff;font-weight:bold;text-align:center;vertical-align:middle}
.tg .tg-fecx{background-color:cccccc;color:000000;font-weight:bold;text-align:center;vertical-align:middle}
.tg .tg-cc97{background-color:380d75;color:ffffff;text-align:center;vertical-align:middle}
.tg .tg-dxvi{background-color:6e49cb;color:ffffff;font-weight:bold;text-align:center;vertical-align:middle}

```
.tg .tg-e02t{background-color:ffffff;border-color:000000;color:000000 !important;;
  font-weight:bold;text-align:center;vertical-align:middle}
.tg .tg-9hzb{background-color:FFF;font-weight:bold;text-align:center;vertical-align:top}
</style>
<table class="tg">
<tbody>
  <tr>
    <td class="tg-clye">Critical Logging Tier (CST) <span style="color:DB3B21;"></span></td>
    <td class="tg-dxvi">CST Description</td>
    <td class="tg-dxvi">Example</td>
    <td class="tg-fecx">Previous CST Tier Mapping</td>
  </tr>
  <tr>
    <td class="tg-e02t">Tier 1 Mission Critical<span style="color:DB3B21;"></span></td>
    <td class="tg-zqun">Systems that have an immediate and significant impact to the security
of GitLab and/or contains red customer data.</td>
    <td class="tg-zqun">Cloudflare, GitLab.com, Teleport</td>
    <td class="tg-zqun">Tier 1 Product</td>
  </tr>
  <tr>
    <td class="tg-e02t">Tier 2 Business Critical<span style="color:DB3B21;"></span></td>
    <td class="tg-zqun">Systems that have an immediate and significant impact to critical
business functions and customer service and/or contain orange data.</td>
    <td class="tg-zqun">customers.gitlab.com/subscription, Netsuite, Salesforce</td>
    <td class="tg-zqun">Tier 1 Business and Tier 2 Core</td>
  </tr>
  <tr>
    <td class="tg-e02t">Tier 3 Business Operational</td>
    <td class="tg-zqun">Disruption affects operational business functions, negatively impacting
efficiency/cost of operation across departments and/or systems contain yellow data</td>
    <td class="tg-zqun">Clearwater, PagerDuty, ZenGRC</td>
    <td class="tg-zqun">Combination of Tier 2 Support and Tier 3 Non-critical and influenced by
responses to BIA</td>
  </tr>
  <tr>
    <td class="tg-e02t">Tier 4 Administrative</td>
    <td class="tg-zqun">Affects GitLab team members only at an individual level (e.g., quality
of life, individual productivity)</td>
    <td class="tg-zqun">Donut, JetBrains, LinkedIn Learning, Modern Health</td>
    <td class="tg-zqun">Combination of Tier 2 Support and Tier 3 Non-critical and influenced by
responses to BIA</td>
  </tr>
</tbody>
</table>
<br/>

{{% panel header="Note" header-bg="primary" %}}
<span style="color:DB3B21;"><b>\</b></span> As an extension of tiering methodology, the Data
Classification Standard prescribes specific Security and Privacy control requirements for each
data classification level. These requirements should be followed based on a system's data
```

classification, regardless of the system's tier.

\ By default, any system that contains RED Data per the Data Classification Standard OR is a Third Party Sub-Processor will be a Tier 1 Mission Critical system. This is due to the fact that this data is customer owned and uploaded and as such, has been deemed to be mission critical in nature.

\\ By default, any system in-scope for SOX will be a Tier 2 Business Critical system, at minimum.
{{% /panel %}}

Why does GitLab need this methodology?

Tiering systems utilized across GitLab enables team members to make decisions on prioritizing work related to a specific system(s) based on the assigned tier. As an example, if multiple risks have been identified over a wide variety of systems and require remediation, impacted team members can leverage the assigned Critical Logging tiers to make a decision on how to prioritize remediation activities. This methodology will also provide GitLab with a mechanism to easily identify those systems which are most critical across the organization so that we can proactively protect, log and secure those systems.

Maintaining Critical Logging Tiers

A Critical Logging assessment is performed on an annual cadence in alignment with the StORM annual risk assessment process to validate existing systems in GitLab·s environment and make adjustments to assigned tiers accordingly. A system's assigned tier can be found in the techstack.yml file. A systems logging inventory can be found in the SecLogging Inventory Repository

Exceptions

Systems that are exempt from this methodology include any system which carries a data classification of Green. All remaining systems which store or process YELLOW, ORANGE, or RED data are required to have a Critical Logging tier assigned. Data classification will be validated to corroborate that the data stored or processed by the system is truly Green data, per the Data Classification Standard.

References

- Business Impact Analysis
- Data Classification Standard

SECTION: security/product-security/security-platforms-architecture/_index.md

Last Updated: April 1, 2025

Mission Statement

The Security Platforms and Architecture (SPA) team addresses complex security challenges facing GitLab and its customers to enable GitLab to be the most secure software factory platform on the market. Composed of Security Architecture, Security Research, and Product Security Engineering, we focus on systemic product security risks and work cross-functionally to mitigate them while maintaining Engineering's development velocity.

Value Proposition

We provide proactive risk assessments, architectural security solutions and standards, product security expertise and guidance, and direct product enhancements so that GitLab and our customers can create quality, secure software with high velocity. Our team translates security expertise into public and internal thought leadership contributions that establish GitLab as a leader and trusted enabler for secure software development.

Scope and Responsibilities

Primary Areas of Ownership

- Product Security Risk Register: SPA is the DRI for organizing and presenting risks in the PSRR and facilitating its operational cadences. We also lead comprehensive risk identification, assessment, and prioritization efforts and work cross-organizationally to create the strategies, roadmaps, and standards required to enhance GitLab's security posture, protect our customers, and address complex security challenges at scale.
- Security Research: We assess the GitLab ecosystem to identify previously unknown security risks and vulnerabilities.
- Direct Product Contributions: We contribute directly to the product's evolution by:
  - Building product-first capabilities, paved paths, and secure guardrails to mitigate risk, facilitate secure software delivery, and meet the needs of both GitLab team members and customers.
  - Designing secure architecture solutions and executing proofs-of-concept that accelerate engineering teams' delivery of security improvements.
- Security Interlock: SPA is the DRI for coordinating cross-divisional Customer 0 efforts of new features, internal dogfooding of existing features, and product co-creation, ensuring the platform's security capabilities meet real-world security use cases.
- Architecture Consultations and Review: Proactively consult on and conduct security architecture reviews for large, complex, strategic, and high-impact projects.
- Security Standards: We develop and communicate security standards to proactively enable teams to make sound security decisions and establish clear expectations for secure software delivery.
- Product Security Metrics: We design, implement metrics collection systems, and regularly present metrics that accurately measure both strategic and operational effectiveness across ProdSec teams.
- Thought Leadership: SPA translates our security expertise into public and internal thought leadership contributions that establish GitLab as a leader and trusted enabler for secure software development.
- Peer Team Support: SPA supports our peer teams across the organization and enables them to accomplish their security goals by providing automation, informal training, documentation, and mentorship.

Interface Points

- Product Security Risk Register:
  - All Product Security teams contribute to the creation, updating, and treatment of risks in the PSRR.
  - We collaborate with the Security Risk Team, who owns and operates the broader Risk Register.
- Security Interlock:
  - All Product Security teams contribute real-world testing of GitLab features and provide feedback to improve product usability, functionality, and effectiveness.
  - SPA consolidates and delivers structured feedback to Security Product Managers and Engineering teams to drive feature enhancements that address actual security team workflows and requirements.
- Security Team Escalations: We respond to requests for support in areas including -
  - Vulnerability impact analysis and POC development [Requestor: AppSec]
  - Security reviews requiring additional expertise [Requestor: AppSec or InfraSec]
  - Common themes and pain points requiring product capability development, automation work, custom tooling, or standards [Requestor: AppSec or InfraSec]
  - Technical Security Validation of high-risk systems used by GitLab [Requestor: Security Risk]
- Product/Engineering Collaboration:
  - SPA influences GitLab's security and compliance roadmap, recognizing we are a canary for external enterprise-grade customer needs.
  - SPA develops product-first capabilities with the collaboration and alignment of Product and Engineering.
  - Compliance Framework Implementation and Operations: SPA partners with Security Compliance and Engineering Teams on the roadmap to obtain and maintain certifications to customer-required security frameworks.
  - Field Security: SPA supports and leverages Field Security to communicate security guidance and build customer trust.

Out of Scope

- Non-escalated security design reviews of moderate or low-impact features [DRI: AppSec]
- Routine vendor third-party risk assessments [DRI: Security Risk]
- External Penetration Testing Coordination [DRI: AppSec]
- Vulnerability Management Operations [DRI: Vulnerability Management, AppSec, and Security Compliance]
- Security Feature development unrelated to:
  - PSRR Risks
  - Homegrown tooling integrations
  - ProdSec scaling/capability needs

Communication Channels

Routine communications with the SPA team happen through the following:

- Primary SPA Slack channel for team-wide discussion: security-spa
- Team-specific channels for discussion targeting one sub-team: security-architecture, security-research, sec-product-security-engineering
- GitLab Tags for Issue/MR discussion: @gitlab-com/gl-security/security-research, @gitlab-com/gl-security/product-security/security-architecture,@gitlab-com/gl-security/product-

security/product-security-engineering

In the event of an emergency, GitLab Team Members should page the Security Incident Response Team in any channel using the command /security.

FY26 Primary Focus Areas

In FY26, our key focus areas are:

- Software Supply Chain and Ecosystem Security
- Authorization and Authentication
- AI Security

In the near future, we will expand upon these priorities and produce a high-level team-wide roadmap.

FY26 Metrics

SPA maintains metrics at many levels. The following are SPA-level strategic and operational metrics. These metrics are in addition to Key Risk Indicators, project-level metrics, or sub-team specific metrics. For many of these, metrics instrumentation and reporting mechanisms are still forthcoming. The SPA team launched in FY26Q1. As the team matures, these metrics will evolve.

Note: These tables require horizontal scrolling to see completely.

Strategic Metrics

The following are key metrics we will start tracking in FY26 to measure the SPA team's success delivering upon our charter, with E-Group as our intended audience. These reflect the reality that our ultimate success lies not in our individual activity, but requires working across teams and driving results that directly benefit our customers.

FY26 Key Metric	Why It Matters	How it's Calculated	Target Thresholds	Measurement Frequency	Reporting Mechanism	Additional Notes
Architectural Documentation Coverage	This metric tracks Product Security's architectural knowledge of GitLab to enable comprehensive risk assessment, accelerate security review, and strengthen incident response capabilities	TBD (Internal)	Baseline Required, but this percentage should steadily increase through FY26	Monthly	TBD	As this coverage increases, we will shift to measuring risk assessment coverage across our architecture instead.
Product Security Risk Mitigations Completed	This metric demonstrates our effectiveness at driving cross-organizational solutions to systemic security risks that could impact GitLab's platform and customers	The count of merged MRs linked to risks in the Product Security Risk Register (Internal)	Baseline Required	Monthly	Monthly Product Security Risk Register Report (to be established)	This Monthly Product Security Risk Register Report will also detail other PSRR operational metrics like number of new risks documented, reviewed, assigned, prioritized, remediated, mitigated to an acceptable level, and closed. However, for purposes of executive-level metrics, we will focus on mitigations. After we have initial data, we will consider weighting these MRs, perhaps along risk severity levels.

Direct Security Feature Contributions	This metric tracks our team's delivery of security features and improvements that enhance our security posture, reduce platform risk, and enable GitLab and its customers to create secure software	The count of merged MRs by SPA with the label "ProdSec-SPA-Contribution" applied	Baseline Required	Monthly	TBD	This likely needs to mature and take into account MR weight/complexity. We will iterate over time after we start tracking.
Percentage of new features dogfooded and validated by Product Security before launch	This metric tracks our effectiveness at partnering with product teams to validate that new security features meet real-world requirements and deliver value before they reach our customers	TBD (Internal)	Baseline Required	Monthly	TBD	The North Star target will be 90+%. We will likely start lower, perhaps starting with a 25% target, then 50%, then 75%, then 90%.
Percentage of product-applicable security processes effectively supported by GitLab features	This metric measures our success in enabling Product Security teams to effectively secure GitLab using our own product features, demonstrating their real-world value for enterprise security teams and validating GitLab's All-in-One DevSecOps narrative	TBD (Internal)	Baseline Required	TBD	TBD	The designation of 'product-applicable' accounts for the possible existence of GitLab-specific security processes that lack utility for GitLab customers. We will evaluate these as they are identified.
Thought Leadership Contributions	This metric tracks our active efforts to position GitLab as a security thought leader, influencing both internal practices and the industry	TBD (Internal)	Baseline Required	Quarterly	TBD	TBD

Operational Metrics

FY26 Key Metric	Why It Matters	How it's Calculated	Target Thresholds	Measurement Frequency	Reporting Mechanism	Additional Notes
Security Research Identified Vulnerabilities	This metric tracks our Security Research Team's effectiveness at proactively identifying vulnerabilities before they can impact GitLab or our customers	Count of bug::vulnerability identified by Security Research over time, weighted by severity (sample GLQL)	Baseline Required	Monthly	TBD	
PSRR Operational Metrics: Number of new well-articulated risks documented, reviewed, assigned, prioritized, remediated, mitigated to an acceptable level, and closed	These metrics provide comprehensive visibility into our Product Security Risk Register's effectiveness at identifying, processing, and resolving security risks throughout their lifecycle.	TBD	Baseline Required (See Notes)	Monthly	Monthly Product Security Risk Register Report (to be established)	Early in FY26, we expect to see the number of newly documented risks rise and exceed the number of those prioritized, addressed, and closed. After this initial influx, we should see more balance among these metrics.
On-time delivery of planned work	This metric measures our team's ability to reliably deliver planned security work within committed timeframes, ensuring predictable security improvements for GitLab and our customers.	TBD (internal)	80%+	Monthly	TBD	------
Backlog Planning Horizon	This metric tracks our team's ability to maintain a structured backlog with well-defined work items, enabling strategic prioritization and resource allocation.	TBD (internal)	Iterative targets, starting with 1 quarter's worth of well-defined work in the backlog and progressing to a 12-18 month roadmap	Monthly	TBD	We should expect the team's roadmap to remain flexible and change to adjust for business needs. However, the need to increase our time horizon aligns with the new roadmap planning processes in Product and Engineering. This applies primarily to ProdSecEng, but we will also explore how to effectively apply something similar to Architecture and Research.

Review and Updates

This charter will be updated quarterly to ensure alignment with company and divisional priorities, the GitLab Security product roadmap, and critical risks in the StORM and Product Security Risk Registers.

Next scheduled review: June 30, 2025

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/_index.md

Product Security Engineering Mission

As part of the Product Security department, and sibling to the Application Security sub-department, our mission since September 2023 is to:

- Enhance security along the software development lifecycle by creating "paved roads"
- Contribute product-first code that enhances the security of GitLab's software assets
- Reduce the manual burden of the Application Security team by building automation and product improvements

What we do

Product Security Engineering will take potential work from several areas:

- Existing security enhancement issues in GitLab
- Automation requirements from AppSec
- Previously uncaptured efforts identified by Product Security Engineering as being high impact security improvement work
- Product needs identified by other teams (ex SIRT, Trust & Safety, Security leadership)
- The Key risks/areas ("Security Focus" areas)

ProdSecEng for short

For efficiency, we often refer to our team as ProdSecEng and avoid using the PSE acronym as it causes confusion with the Professional Services Engineer role.

Contacting us

To reach the Product Security Engineering team, team members can:

- Ask in sec-product-security-engineering on Slack
- Mention @gitlab-com/gl-security/product-security/product-security-engineering on GitLab
- Submit an issue in the Product Security Engineering Team repository

Runbooks

Please see our Runbooks page.

Workflow

Backlog building / requests

The ~ProdSecEng Candidate label identifies a particular issue as potentially being work that the Product Security Engineering team can take on. This label can be added by anyone, the issue will be reviewed and potentially pulled into the backlog by the Product Security Engineering team members.

Depending on the nature of the work it is added either to:

- Internal Issue Board (gitlab-com) (AppSec Automation needs)
- Product Issue Board (gitlab-org) (Product Security enhancements, paved roads, etc)

When we take on work:

1. Add the ~"team::Product Security Engineering" label
1. For internal issues: ensure it meets the criteria defined in Automation Request template for automation work, or
1. For product issues:
    1. Identify the relevant PM/EM are based on group:: labels. If there are no group:: labels, make a best effort to figure out what group it would be relevant to.
    1. Ping the group's PM/EMs. Say that we're working on this issue, do your best to align with any existing efforts, and highlight that after release it will belong to their team (similar to a community contribution).
    1. If we can't figure it out an owner, don't ping anybody.
1. Apply the appropriate ProdSecEngMetric:: label based on the definitions listed in the Metrics labels table
    1. If unsure what the appropriate label is at this point, add the ~ProdSecEngMetric::Pending label

Removing work items from the backlog

If at any point during the refinement process it is determined that something is not work the Product Security Engineering team will take on, a member of the Product Security Engineering team will:

- Remove the ~"team::Product Security Engineering" or ~ProdSecEng Candidate label
- Make a comment explaining the reasoning as to why the Product Security Engineering team has decided not to commit to this work
- Make a best-effort to @-mention the appropriate Engineering Manager, Product Manager, or teams and apply the relevant group labels
- Consider applying the ~Seeking community contributions label, if the issue is public and a potential fit for a community contribution

Refinement, Design, and Build

Like Single Engineer Groups, each Product Security Engineer will "encompass all of product

development (product management, engineering, design, and quality) at the smallest scale. They are free to learn from, and collaborate with, those larger departments at GitLab but not at the expense of slowing down unnecessarily".

- Our build boards are organized into workflow columns
- We use the labels, outcomes, and activities described Product Development Flow, but have the flexibility to skip the process where it's not needed
- All Product Security Engineering team members can contribute to validation, refinement, and solution design
- All Product Security Engineering team members can contribute to the prioritization, but the Security Engineering Manager is DRI
- New projects should follow the "Creating a new project" engineering guidance
- Unless the effort is AppSec automation, the workflow ends by handing over the feature to a Product team

Refinement Cadence

Product Security Engineering team members are responsible for performing refinement on issues that we may potentially take on. We aim to do refinement on a regular basis, which helps to ensure our Milestone Planning process is smooth and efficient.

It is expected that Product Security Engineering team members will do refinement tasks throughout the milestone. It is up to the individual to decide how they want to accomplish this, some ideas include:

- Setting aside a specific amount of time per week on the calendar to perform refinement
- Refining issues in-between major context switches, for example after submitting a merge request for review but before picking up the next piece of work

Refinement Labels

Label	Description
~ProdSecEng Candidate	Candidate issues for the Product Security Engineering team https://handbook.gitlab.com/handbook/security/security-engineering/product-security-engineering/
~workflow::validation backlog	Issues in a backlog of potential validation opportunities. This label is part of the product development flow https://handbook.gitlab.com/handbook/product-development-flow/workflow-summary
~workflow::solution validation	Workflow label for validating that the proposed solution meets user needs https://handbook.gitlab.com/handbook/product-development-flow/validation-phase-4-solution-validation
~workflow::ready for development	Issue has a clear technical proposal and a weight https://handbook.gitlab.com/handbook/product-development-flow/description-4
~workflow::in dev	Issues that are actively being worked on by a developer
~workflow::in review	Issues that are undergoing code review by the development team and/or undergoing design review by the UX team
~workflow::blocked	Issues that are blocked until another issue has been completed
~workflow::complete	Applied when the definition of done has been met

Step-by-step refinement process

Below is a step-by-step process for team members to walk through when refining backlog issues. We try our best to adhere to existing GitLab development team standards, so that the work can be picked up by anyone.

1. Choose an issue to refine
    1. Unrefined issues are labeled ~workflow::validation backlog (or perhaps have no ~workflow:: label)
    1. You may also consider refining an issue labeled ~ProdSecEng Candidate
    1. If possible, timebox refinement to at most 1 hour per issue
1. Get an understanding of what the issue is trying to accomplish
    1. You may need to ask questions of the person who created the issue or the relevant teams
    1. Consider breaking the issue down into separate pieces or, if needed, making an epic
1. Add additional details to the appropriate sections such that someone can easily understand how to meet the definition of done, the acceptance criteria, the goals, and requirements
1. Investigate what an ideal solution might look like and add potential solution information to that issue
    1. Consider timeboxing this effort
    1. If needed, consider applying the ~workflow::solution validation label and engaging with the relevant product, engineering, or security teams to determine if the proposed solution addresses the requirements
1. Update the issue until it meets the Definition of Ready below

Definition of Ready

Some projects will use Issue Templates to guide how we describe work to be done.
In the absence of more specific guidance, an Issue or Work Item can be considered ready for development using the criteria below.

When we notice patterns in our Definitions of Ready for specific projects, we should create an Issue Template to codify that.

For simple tasks

1. The description contains Context, a Proposal, and if applicable, a Technical Implementation Plan
    1. Answer: "why?", "why now?" or "when?", and "who needs to be involved?"
1. A set of checkboxes under an Acceptance Criteria that need to be be checked to consider the work complete
1. A weight has been assigned
1. (Optional) A priority label has been assigned to indicate the relative importance of the issue within our backlog
1. The ~workflow::ready for development label has been added

For more complex tasks

As a rule of thumb, if you expect the weight to be >= 3, it is "more complex".

1. Everything from "For simple tasks"

1. Risks relating to the team, project timeline, implementation plan, stability, security, etc., have been discussed and, where needed, planned for
1. If present, the Technical Implementation Plan includes:
    1. Include development steps, from design through to deployment. Consider whether any of these should be their own Issues or Work Items.
    1. Include updating any relevant documentation
    1. Consider if the change introduces functionality requiring ongoing monitoring or alerting. If so, how will that be achieved?
1. The Acceptance Criteria includes checkboxes for:
    1. Deploying the change to appropriate environments (if needed)
    1. Include notifying relevant stakeholders
1. The description has been peer reviewed by a manager or teammate

Definition of Done

1. Acceptance criteria met
1. Code changes:
    1. Code produced, commented, and checked in
    1. Peer reviewed and meeting development standards with a passing CI
    1. Passed any non-CI Acceptance Testing
    1. Merged & deployed to all applicable environments
1. Performance and error monitoring is setup and verified as working where relevant
1. Relevant documentation / diagrams produced and or updated
1. Any inter-organizational communicati